

READING MATERIALS

Activity 4.3.1: Input/Output Architecture

Concept of Input/output

Input and output (I/O) in computer architecture refer to the mechanisms and processes through which a computer system interacts with external devices or users to receive data (input) or transmit data (output).

Input Input refers to the data or instructions that are received by a computer system from external sources.

Output Output refers to the processed or transformed data that is sent from the computer system to external devices or users.

I/O Operations

I/O operations involve transferring data between the computer system and external devices. I/O operations typically include the following steps:

- **Issuing an I/O request:** The computer system initiates an I/O request by sending a command to the appropriate I/O device, specifying the type of operation (e.g., read or write) and the location of the data.
- **Data transfer:** The I/O device transfers data between the computer system and the external device. This can be a one-way transfer from the external device to the computer system (input) or vice versa (output), or a bidirectional transfer for devices that support both input and output.
- **Interrupt handling:** Once the I/O operation is complete, the I/O device generates an interrupt to inform the CPU that the operation has finished. The CPU then interrupts the normal program execution and handles the interrupt by executing an interrupt service routine (ISR) to process the data received from the I/O device.

I/O Techniques

Computer systems use various techniques to manage I/O operations efficiently, including programmed I/O, interrupt-driven I/O, and direct memory access (DMA).

- **Programmed I/O:** In programmed I/O, the CPU directly controls the I/O operations by polling or busy-waiting for the completion of the I/O operation. This can be inefficient as the CPU is tied up in waiting for I/O to complete, resulting in wasted CPU cycles.
- **Interrupt-driven I/O:** In interrupt-driven I/O, the CPU initiates an I/O operation and then continues executing other tasks. When the I/O operation is complete, the I/O device generates an interrupt to the CPU, which then handles the interrupt and resumes normal execution.
- **Direct memory access (DMA):** DMA is a technique that allows the I/O device to directly transfer data to or from the computer's memory without CPU intervention. DMA reduces CPU overhead and improves I/O performance by offloading the data transfer tasks from the CPU to the I/O device.

Interrupt Handling

Interrupt handling is a critical process in computer systems that allows the system to respond to events or signals from external devices or internal conditions in a timely and efficient manner.

An interrupt is a signal that interrupts the normal flow of a computer's central processing unit (CPU) to temporarily suspend the execution of the current instruction or process and transfer control to a special routine called an interrupt handler or interrupt service routine (ISR).

Key steps involved in interrupt handling:

- **Interrupt Detection:** The first step in interrupt handling is detecting the occurrence of an interrupt. This can be triggered by external events, such as input from a keyboard and mouse.
- **Interrupt Request (IRQ):** When an interrupt event occurs, the corresponding device or component sends an interrupt request (IRQ) signal to the CPU to indicate that an interrupt has occurred and needs attention.
- **Interrupt Vectoring:** The CPU identifies the source of the interrupt through a process called interrupt vectoring. Interrupt vectoring involves mapping the interrupt request (IRQ) to a specific interrupt handler routine, usually stored in a designated location in memory, based on a unique interrupt number or code associated with the interrupt event.

- **Saving Context:** Before transferring control to the interrupt handler routine, the CPU saves the context of the current process or instruction being executed, including register values, program counter, and other relevant CPU state information. This allows the system to resume the interrupted process later when the interrupt handling is complete.
- **Interrupt Handling:** The CPU transfers control to the designated interrupt handler routine (ISR), which is responsible for handling the interrupt event. The ISR performs the necessary tasks such as processing I/O operations, servicing hardware or software requests, updating system state, or handling errors.
- **Clearing Interrupt:** Once the interrupt has been serviced, the interrupt is cleared, and the CPU resumes normal operation by restoring the saved context and returning control to the interrupted process.
- **Interrupt Prioritization:** In systems with simultaneous interrupts, prioritization determines the order of handling based on priority levels. Higher priority interrupts are addressed first.
- **Error Handling:** Routines may include mechanisms to detect and handle errors occurring during the interrupt process.
- **Interrupt Masking:** This involves temporarily disabling interrupts during critical operations to avoid interference or conflicts.

Activity 4.3.3: Direct Memory Access (DMA)

DMA stands for Direct Memory Access. It is a computer architecture feature that allows data to be transferred between peripheral devices and main memory without involving the central processing unit (CPU).

How DMA Works In a typical system without DMA, the CPU handles all data transfers, which increases utilization and reduces performance. With DMA, a separate controller manages these transfers independently.

Key Components and Steps:

- **DMA Controller:** A separate hardware component responsible for managing data transfer. It is programmed with source/destination addresses and transfer size.

- **DMA Request:** The peripheral device sends a request to the DMA controller with transfer parameters.
- **DMA Transfer:** The controller initiates the transfer directly between the device and memory. This can be done in parallel with the CPU's execution of other instructions.
- **CPU Involvement:** During the transfer, the CPU is free to perform other tasks, improving system performance.
- **Interrupts:** Once the transfer is complete, the DMA controller generates an interrupt to notify the CPU.